

ResumelQ

Real-World Product and AI/ML Roadmap

A practical record of what to improve before public launch and how to turn the project into a strong placement portfolio piece.

Core product direction

ResumelQ should become a career-preparation workspace: resume analysis, job matching, personalized practice, mock tests, AI interviews, progress tracking, and job-search support.

How to use this document

- ☐ Choose one task at a time.
- ☐ Do not start model training before the core product is reliable.
- ☐ Mark each item complete only after testing it with a real user flow.
- ☐ Keep screenshots, metrics, and notes for your portfolio and resume.

1. The main decision: do not train a large model yet

The best path for ResumeIQ is a hybrid system. Keep a strong external language model for generation, but build your own domain-specific scoring, skill extraction, matching, evaluation, and product logic. This gives you real ML work without spending months training a general-purpose model.

Recommended architecture

Resume PDF -> text extraction -> skill normalization -> embeddings and matching -> your scoring engine -> LLM explanation and rewriting -> saved report and analytics.

Now	Stability	Security, validation, privacy, error handling, tests, deployment readiness.
Next	Domain intelligence	Skill taxonomy, deterministic resume/job scoring, embeddings, evidence-based reports.
Later	ML model	Train a small readiness/match model using labeled examples.
Optional	Fine-tuning	Use LoRA on a small open model only after you have quality data and evaluation.

Why not train from zero?

- [] Training needs large data, GPUs, evaluation, monitoring, and model-serving infrastructure.
- [] A custom model does not remove cost; it replaces API cost with GPU hosting and maintenance.
- [] For placement, a measured end-to-end AI system is more impressive than an unfinished foundation model.
- [] LoRA/PEFT can be explored later because it adapts a pretrained model with fewer trainable parameters.

2. P0: must fix before real public use

These items affect security, trust, privacy, or whether the product behaves correctly. Finish them before marketing the site publicly.

Authentication

Use secure httpOnly cookies or document the token strategy carefully. Align the two token names used by the frontend. Remove token and decoded-user logs. Increase password requirements.

Email verification

The README describes verification, but the backend registration flow currently returns a token immediately. Choose one behavior and make registration, login, OTP, and frontend routes consistent.

Production CORS

Replace origin '*' with the real frontend domain. Review credentials, allowed methods, headers, and environment configuration.

Rate limiting

Protect login, registration, OTP, report generation, mock-test generation, and interview-answer endpoints. Add per-user AI quotas.

File security

Validate MIME type and extension, reject malformed PDFs, limit request size, handle parser failures, and never trust a client filename.

Privacy

Add consent, retention rules, delete-account, delete-resume, and recording controls. Tell users what happens to resume, audio, and video data.

Assessment and interview correctness

- [] Complete or remove the proctoring warning logic; currently the warning function exits before its main logic.
- [] Prevent duplicate submissions and resume interrupted sessions safely.
- [] Show a clear state when AI generation fails, times out, or reaches quota.
- [] Test camera, microphone, fullscreen, browser refresh, back navigation, and mobile behavior.
- [] Ask for recording consent before audio/video capture and show a visible recording indicator.

3. P1: make the current product feel professional

The main product loop is already valuable. Improve the clarity and confidence of each step so a first-time user always knows what to do next.

Dashboard

- [] Show one primary next action: Continue preparing for [target role].
- [] Add readiness score, weakest skill, recent score trend, and recommended practice.
- [] Show a profile selector clearly and keep all reports separated by profile.
- [] Add empty states with useful next steps, not only 'no data' messages.

Profile creation

- [] Convert the long form into a 4-6 step wizard with progress indicator.
- [] Add file validation, upload progress, character counters, examples, and save draft.
- [] Add a sample profile/demo mode so visitors can understand the product without entering personal data.

Reports

- [] Separate Overview, Resume Match, Skill Gaps, Questions, Practice Plan, and History into clear tabs.
- [] Explain every score with evidence from the resume and job description.
- [] Show before/after bullet rewrites and let users copy or save suggestions.
- [] Keep PDF export complete, multi-page, branded, and tested on long reports.

Reliability and accessibility

- [] Add a global error boundary, retry buttons, skeleton loaders, and session-expired handling.
- [] Fix lint warnings, unused imports, effect dependencies, and debug output.
- [] Test keyboard navigation, focus states, labels, contrast, and screen-reader messages.
- [] Test 360px mobile, tablet, and desktop layouts.

4. P2: build your own ResumeIQ intelligence layer

This is the best way to make the project genuinely relevant to machine learning without trying to recreate a general chatbot.

A. Skill extraction and normalization

- [] Create a skill taxonomy for frontend, backend, data, cloud, testing, tools, and soft skills.
- [] Normalize aliases such as JS -> JavaScript, ReactJS -> React, and Mongo -> MongoDB.
- [] Extract skills separately from resume, job description, and interview answers.
- [] Store extracted skills with confidence and source text so the report is explainable.

B. Resume-job matching

- [] Calculate exact skill overlap and missing required/preferred skills.
- [] Generate embeddings for resume sections, job requirements, and skills.
- [] Calculate semantic similarity using cosine similarity.
- [] Combine skill, experience, keyword, project, education, and semantic signals into a weighted score.
- [] Show the user exactly which evidence increased or reduced the score.

C. Small ML model

- [] Collect 100-300 human-reviewed resume/job examples before training.
- [] Label match score, missing skills, seniority fit, and interview readiness.
- [] Start with Logistic Regression, Random Forest, or XGBoost.
- [] Measure precision, recall, mean absolute error, and agreement with human reviewers.
- [] Use the LLM to explain the score, not to secretly decide every score.

D. RAG and trusted knowledge

- [] Create a knowledge base of role expectations, interview rubrics, STAR guidance, and skill definitions.
- [] Retrieve relevant guidance before generating feedback.
- [] Cite the relevant rubric or evidence inside the report where possible.

5. P3: features that can make ResumeIQ stand out

Tailored resume workspace

Create job-specific resume versions, rewrite bullets for impact, generate summaries, create cover letters, and compare versions.

Real coding assessment

Add editor, language selection, sample tests, code execution, hidden tests, complexity feedback, and coding score.

Adaptive interview

Ask follow-up questions based on the answer, adjust difficulty, and provide technical and behavioral rubrics.

STAR feedback

Score Situation, Task, Action, and Result separately and show a stronger answer structure.

Progress analytics

Show skill mastery, score trends, practice streak, weekly goal, and recommended next practice.

Job application tracker

Save companies, roles, job descriptions, resume versions, interview dates, stages, and notes.

Study plan

Turn skill gaps into daily tasks, spaced repetition, reminders, and completion tracking.

Shareable report

Create a privacy-safe, read-only report link or downloadable portfolio summary.

Interview quality upgrades

- [] Score answer relevance, technical correctness, structure, clarity, and communication.
- [] Analyze speaking speed and filler words only with explicit consent.
- [] Show transcript, feedback, suggested answer, and next practice action.
- [] Allow text-only interview mode for users without camera or microphone access.

6. Engineering and placement readiness

A real public project needs proof that it is maintainable, testable, and safe—not only visually attractive.

Engineering checklist

- ☐ Add unit tests for scoring, skill normalization, report sanitization, and result calculations.
- ☐ Add API integration tests for authentication, profiles, reports, mock tests, and interviews.
- ☐ Add end-to-end tests for the main user journey.
- ☐ Add request validation with Zod on every backend endpoint.
- ☐ Add centralized structured logging and error monitoring.
- ☐ Add pagination for reports, questions, sessions, and history.
- ☐ Add database indexes and cleanup rules for old sessions and generated audio.
- ☐ Add frontend route lazy loading to reduce the large initial bundle.
- ☐ Remove duplicate/stale router paths and document the single active router.

Deployment checklist

- ☐ Deploy frontend and backend separately with production environment variables.
- ☐ Use HTTPS and a real domain.
- ☐ Configure production CORS and secure cookies.
- ☐ Set up database backups and a health check.
- ☐ Add CI/CD: install, lint, build, test, and deploy on every pull request.
- ☐ Add monitoring for latency, error rate, AI quota failures, and upload failures.

Placement presentation

- ☐ Create an architecture diagram.
- ☐ Record a 2-minute product demo.
- ☐ Show before/after screenshots and real test results.
- ☐ Mention the hybrid architecture, custom scoring, embeddings, and evaluation process.
- ☐ Use only real metrics; do not invent user counts or accuracy numbers.

7. Recommended 30/60/90-day plan

Days 1-10	Safety	Auth consistency, CORS, rate limits, file validation, privacy, logs, error states.
Days 11-20	UX	Dashboard next action, profile wizard, report evidence, mobile, accessibility, PDF testing.
Days 21-35	Own scoring	Skill taxonomy, normalization, deterministic match score, embeddings, score explanations.
Days 36-50	Practice	Adaptive interview, STAR rubric, assessment reliability, progress analytics.
Days 51-70	Evaluation	Labeled dataset, baseline model, metrics, regression tests, feedback loop.
Days 71-90	Launch	Deployment, monitoring, demo account, pilot users, feedback, portfolio documentation.

Your task log

Use this section as a record. Add a date and one sentence describing what changed and how you tested it.

The best next task

Start with the P0 security and reliability checklist. After that, build skill extraction and resume-job scoring. Do not begin foundation-model training until those two layers are working and measured.

8. Final product direction

ResumeIQ should not be positioned as only an AI resume checker. Position it as a personalized career preparation system that connects a candidate's resume, target role, skill gaps, practice sessions, and interview readiness in one place.

Strong portfolio statement

Built a hybrid AI career platform that combines resume parsing, job-skill matching, custom readiness scoring, semantic embeddings, retrieval-based feedback, AI-generated interview practice, and performance analytics.

Do not measure success by how many AI features you add. Measure it by whether a user can answer these questions:

- ☐ What role am I preparing for?
- ☐ What are my biggest skill gaps?
- ☐ What should I practice today?
- ☐ Am I improving?
- ☐ What should I change in my resume to improve my chances?

One task at a time. One tested improvement at a time. That is how ResumeIQ becomes a real product.